

Group 5 - Jaewon Jang, Jason Tofte, Ross Roach

8/1/2026

Supply Chain Track

MGMT599 Project Proposal

The Business Problem

For our final project, we will be conducting demand forecasting and inventory analytics on a dataset of Walmart retail goods. Walmart routinely faces a decision of how much of each product they should have on hand in each store. If they have too much product, then they have spent money on stock that might have been better spent somewhere else. If they don't have enough product, then they are missing out on potential revenue. Because of this, carrying as close to the needed amount of product is essential for revenue maximization. With our project, we plan to solve this issue with forecasting. The intended user of this data will be whoever is responsible for store inventory levels. They should be able to use this data to make more informed decisions on levels of stock of products, leading to more revenue. This problem is relevant from an analytics perspective because forecasting requires a lot of data and a very technical analysis of it. The principles that executing a project like this require will be transferable to many business problems, therefore making it a relevant one to attempt.

Dataset Plan

This project will use data from an M5 forecasting challenge on Kaggle. This dataset was provided by Walmart, and shows the daily inventory levels of 3,049 different products across 3 store locations. The data is provided in structured csv documents with a row for every item and store combination that serves as a primary key. The main fields of the data that will be relevant to us are the item_id, store_id, state_id and the columns d_1 to d_1941. The column d_1 represents the number of units in inventory at the store on day 1, and d_1941 is the same information of the product on day 1941. The item_id will be useful in identifying the part, and the store_id and state_id will be useful in differentiating forecasts per location. The day columns will be used as an input into our forecast to train our model on the item's past inventory behavior for more effective forecasting.

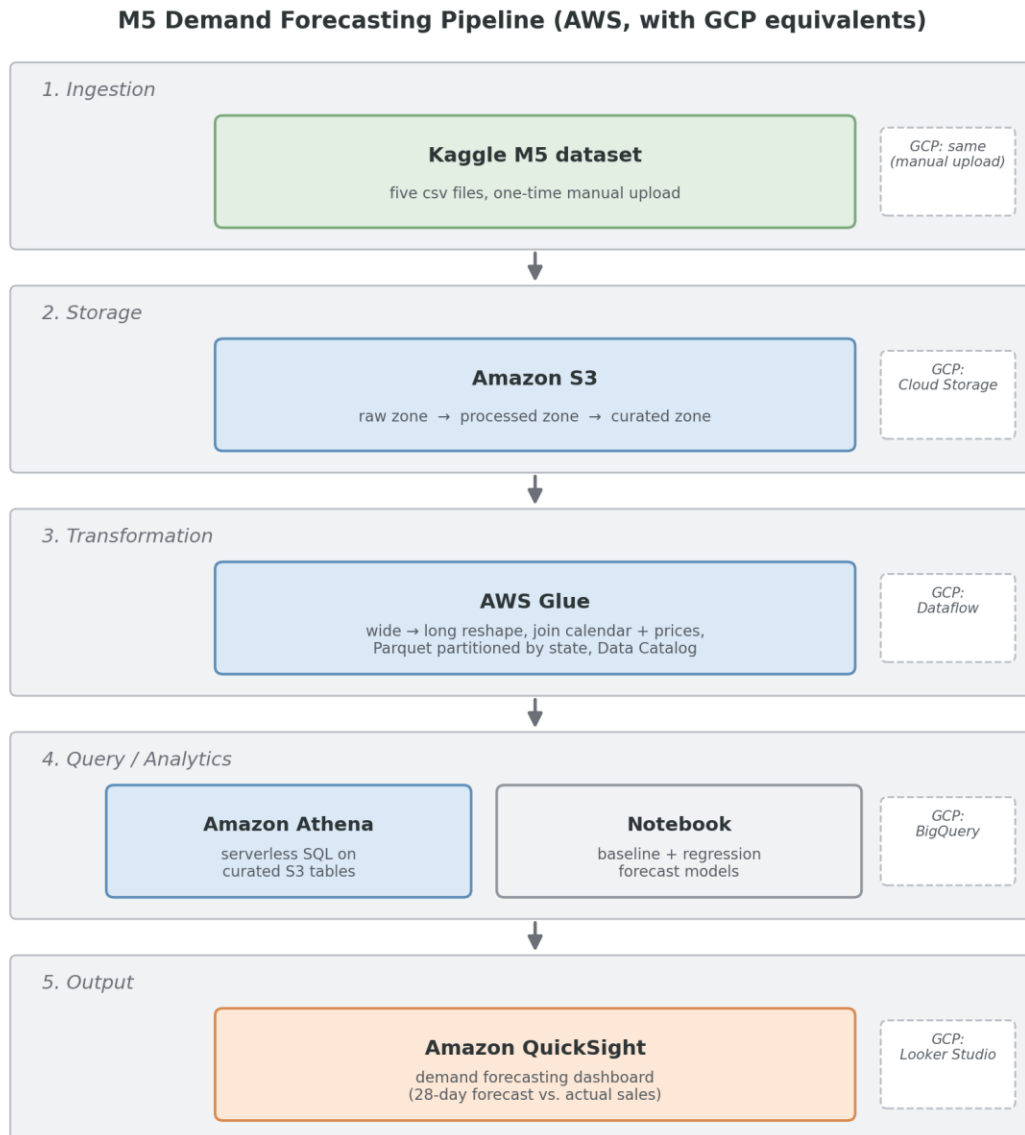
Proposed Cloud Architecture

Our proposed architecture follows a standard data pipeline structure with five layers:

1. Ingestion
2. Storage
3. Transformation

4. Querying and analytics
5. Output

The diagram below shows how data moves through each layer and which AWS service handles each step.



Since the dataset is a fixed set of csv files rather than a live data feed, we will design the pipeline to be as simple as possible while still demonstrating each layer properly. We want avoid adding services the project does not actually need.

For ingestion, we will upload the five csv files from the M5 dataset directly into an Amazon S3 bucket. Because the data is a one-time download from Kaggle rather than a streaming or continuously updating source, a simple upload into a raw data folder is the appropriate ingestion method, and using something like Kinesis would be unnecessary complexity for data that never changes. S3 also serves as our storage layer. We will organize the bucket into three zones: a raw zone holding the original untouched csv files, a processed zone holding the cleaned and reshaped data, and a curated zone holding the final tables our analytics will run against. Keeping the raw files untouched matters because if we make a mistake during transformation we can always go back to the original data and start over without re-downloading anything.

The transformation layer is where most of the technical work happens. The main sales file stores each item and store combination as a single row with a column for every one of the 1,941 days, and that format cannot be queried or modeled effectively. We will use an AWS Glue job to reshape this data so that each row represents one item, at one store, on one day, and then join in the calendar and price information so that every row also carries the day's events and the item's price that week. The output of this job will be written to the processed zone as compressed Parquet files partitioned by state, which matters later for both query speed and cost. We will also use the Glue Data Catalog and a Glue crawler to register the table schemas, which is what allows Athena to query files sitting in S3 as if they were database tables.

Analytics Output

For the querying and analytics layer, we will use Amazon Athena to run SQL directly against the curated data in S3, and will keep a notebook for the forecasting model itself. Athena fits this project well because it is serverless, meaning we are only charged for the queries we run rather than paying to keep a database running around the clock. The output layer will be Amazon QuickSight, which connects to Athena and turns our query results into the dashboard described in the next section. As a brief comparison, the same pipeline could be built on GCP by swapping S3 for Cloud Storage, Glue for Dataflow, Athena for BigQuery, and QuickSight for Looker Studio. The structure of the pipeline would be identical, and BigQuery is a very close equivalent to Athena since both are serverless and charge per query. We chose AWS because it has been the platform used most often in this course, but nothing about our design is specific to one provider.

The final output of our project will be a demand forecasting dashboard along with the SQL analysis and forecasting model behind it. The dashboard will be built in QuickSight and will allow the user to view past daily sales filtered by state, store, and product category, along with our forecast of sales for the next 28 days. We chose 28 days because that was the forecast window used in the original M5 competition, and it also gives us a built in way to measure accuracy by holding out the final 28 days of data and comparing our predictions against what

actually sold. For the person responsible for inventory, this is the view that matters. If the dashboard shows a category in a particular store is expected to sell a certain amount over the next four weeks, that number can be compared against current stock to decide whether an order needs to be placed.

Underneath the dashboard, we will write a set of Athena SQL queries that answer questions like which product categories sell the most at each store, how sales respond when prices change, and how demand shifts around holidays and SNAP benefit days recorded in the calendar file. These queries prove the pipeline works end to end, and they also tell us which variables are worth including in the forecast. We plan to start with a simple baseline, like a seasonal naive forecast that assumes next month will look like the same period last year, and then compare it against a regression model that uses the calendar and price data as additional inputs, evaluating both on the same held out window so the comparison is fair. Our goal is not the most advanced model possible, but a defensible one attached to a working pipeline. The final deliverable package will be the dashboard, the SQL scripts, and a notebook containing the models and their evaluation.

Risks and Challenges

There are several risks and challenges that we may face during this project. The first is data quality and complexity. The M5 dataset contains a large number of products, stores, and daily observations, and the data is spread across multiple files. Errors during the transformation or joining of these files could result in inaccurate analysis or forecasts. To manage this risk, we will keep the original files unchanged in the raw S3 zone and validate the transformed data before using it for analysis.

A second risk is the complexity of the forecasting process. Demand can be affected by factors such as holidays, price changes, and other events, which may make accurate forecasting difficult. We plan to manage this risk by starting with a simple baseline model and then comparing it with a regression model that includes additional variables. We will evaluate both models using the same 28-day holdout period so that we can determine whether the more complex model provides a meaningful improvement.

A third risk is the potential cost and implementation complexity of using multiple AWS services. Services such as Glue, Athena, and QuickSight could create unnecessary costs if they are not managed carefully. We will reduce this risk by using serverless services where appropriate, limiting the amount of data scanned by Athena through Parquet files and partitioning, and avoiding unnecessary services that do not directly support our project goals.

Initial Cost and Cleanup Plan

We will keep the project cost-conscious by using AWS services only when they provide a clear benefit to the project. Since the M5 dataset is a fixed dataset, we will not use continuous streaming services or maintain an always-running database. The data will be stored in Amazon S3, and we will use serverless services such as AWS Glue and Amazon Athena only when needed. Converting the processed data to compressed Parquet files and partitioning it by state will also help reduce the amount of data scanned by Athena and lower query costs. We will also limit the use of QuickSight to the development and demonstration of the final dashboard.

After the project is completed, we will clean up the AWS environment to prevent unexpected charges. We will delete temporary files and unnecessary processed data, remove Glue jobs and crawlers that are no longer needed, and delete any Athena resources or other temporary configurations. We will also remove the QuickSight dashboard and related resources if they are no longer required. Before deleting anything, we will make sure that any files or results needed for the final submission have been downloaded and saved locally. This cleanup process will help ensure that the project does not continue generating cloud costs after the course is completed.